Lab No.

Exception Handling Experiments

1.Write a Java program to demonstrate **exception handling using try and catch** by handling division by zero.

Code:

```
public class expection1 {
    public static void main(String[] args){
        try{
            int a=10;
            int b=0;
            int result=a/b;

System.out.println("Result:"+result);
        } catch (ArithmeticException e) {

System.out.println("Error:division by zero is
not possible");
        }
        System.out.println("Program executed
normally");
    }
}
```

Output:

```
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Users\ITLAB2\Documents\119 Java\src>javac expection1.java

C:\Users\ITLAB2\Documents\119 Java\src>java expection1
Error:division by zero is not possible
Program executed normally

C:\Users\ITLAB2\Documents\119 Java\src>
```

2. Write a Java program to demonstrate different **types of exceptions** such as `ArithmeticException`, `ArrayIndexOutOfBoundsException`, and `NullPointerException`.

Code:

Lab No.

Exception Handling Experiments

```
public class exception2 {
    public static void main(String[] args) {
        try {
            int x = 5 / 0;
        } catch (ArithmeticException e){

System.out.println("ArithmeticException" +
e.getMessage());
        }
        try {
            int[] nums = {10, 20, 30, 40};
            System.out.println(nums[4]);
        } catch
(ArrayIndexOutOfBoundsException e){

System.out.println("ArrayIndexOutOfBoundExcep
tion: Invalid arrayIndex");
        }
        try{
            String a=null;
            System.out.println(a.length());
        }catch(NullPointerException e){
            System.out.println("Null pointer
exception: Invalid String index");
        }
    }
}
```

Output:

```
C:\Users\ITLAB2\Documents\119 Java\src>javac exception2.java

C:\Users\ITLAB2\Documents\119 Java\src>java exception2
ArithmeticException/ by zero
ArrayIndexOutOfBoundException: Invalid arrayIndex
Null pointer exception: Invalid String index

C:\Users\ITLAB2\Documents\119 Java\src>
```

Lab No.

Exception Handling Experiments

3.Write a Java program to demonstrate an **uncaught exception** and observe the default JVM exception message and stack trace.

Code:

```
public class exception3 {
    static void main() {
        int[] numbers={10,20,30,40};
        System.out.println(numbers[10]);
        System.out.println("This line never
executes");
    }
}
```

Output:

```
C:\Users\ITLAB2\Documents\119 Java\src>javac exception3.java

C:\Users\ITLAB2\Documents\119 Java\src>java exception3
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 10 out of
        at exception3.main(exception3.java:4)

C:\Users\ITLAB2\Documents\119 Java\src>
```

4.Write a Java program to handle **multiple exceptions using multiple catch clauses**.

Code:

```
public class exception4 {
    static void main() {
        try{
            int[] arr=new int[3];
            int
choice=Integer.parseInt(args[0]);
            arr[choice]=10/choice;
        }catch(ArithmeticException e){
            System.out.println("Cannot divide
by zero!");
        }catch(ArrayIndexOutOfBoundsException
e){
            System.out.println("Array
indexError");
```

```
        }catch(NumberFormatException e){
            System.out.println("Invalid
number format!");
        }catch(Exception e){
            System.out.println("some other
errors:"+e);
        }
        System.out.println("Program
continues....");
    }
}
```

Output:

```
C:\Users\ITLAB2\Documents\119 Java\src>javac exception4.java
exception4.java:5: error: cannot find symbol
          int choice=Integer.parseInt(args[0]);
                                      ^
  symbol:   variable args
  location: class exception4
1 error

C:\Users\ITLAB2\Documents\119 Java\src>
```

5. Write a Java program to demonstrate the use of the **throw statement** to explicitly generate an exception when a given condition is violated.

Code:

```
public class exception5 {
    static void checkAge(int a){
        if(a<18){
            throw new
ArithmeticException("Not eligible to
voting");
        }
        else{
            System.out.println("Eligible to
vote");
        }
    }
    static void main() {
```

```
        checkAge(17);
        System.out.println("the program
continues");
    }
}
```

Output:

```
:\Users\ITLAB2\Documents\119 Java\src>java exception5
xception in thread "main" java.lang.ArithmeticException: Not eligible to voting
        at exception5.checkAge(exception5.java:4)
        at exception5.main(exception5.java:11)

:\Users\ITLAB2\Documents\119 Java\src>
```

6.Write a Java program to demonstrate the use of the **throws keyword** by propagating an exception to the calling method.

Code:

```
public class exception6 {
    static void divide(int a,int b) throws
ArithmeticException{
        int result=a/b;
        System.out.println("Result:"+result);
    }

    static void main() {
        try{
            divide(10,0);
        } catch (ArithmeticException e) {
            System.out.println("caught in
main:"+e.getMessage());
        }
    }
}
```

Output:

```
C:\Users\ITLAB2\Documents\119 Java\src>javac exception6.java

C:\Users\ITLAB2\Documents\119 Java\src>java exception6
caught in main:/ by zero

C:\Users\ITLAB2\Documents\119 Java\src>
```

7.Write a Java program to demonstrate the use of the **finally block** for executing statements regardless of whether an exception occurs.

Code:

```
public class exception7 {
    public static void main(String[] args){
        try{
            int a=10;
            int b=0;
            int result=a/b;

System.out.println("Result:"+result);
        } catch (ArithmeticException e) {

System.out.println("Error:division by zero is
not possible");
        }
        finally{
            System.out.println("This code
will always executed");
        }
        System.out.println("Program executed
normally");
    }
}
```

Lab No. **Exception Handling Experiments**

Output:

```
C:\Users\ITLAB2\Documents\119 Java\src>javac exception7.java

C:\Users\ITLAB2\Documents\119 Java\src>java exception7
Error:division by zero is not possible
This code will always executed
Program executed normally

C:\Users\ITLAB2\Documents\119 Java\src>
```

8.Write a Java program to create a **user-defined exception** by extending the `Exception` class.
Code:

9.Write a Java program to validate a student's marks and throw a **custom exception** when the marks are outside the valid range.
Code:

10.Write a Java program that combines **try, multiple catch, throw, throws and finally** in a single application.
Code:

Lab No. **I/O streams: The Byte Streams Experiments**

1	Write a Java program to demonstrate the use of **InputStream and OutputStream** to read bytes from an input source and write bytes to an output destination.
2	Write a Java program to **read the contents of a file byte-by-byte** using `FileInputStream` and display the contents on the console.
3	Write a Java program to **write data into a file** using `FileOutputStream` and verify the contents of the file.
4	Write a Java program to **copy the contents of one file to another file** using `FileInputStream` and `FileOutputStream`.
5	Write a Java program to **copy an image from one file to another** using byte streams and handle `FileNotFoundException` and `IOException` using `try-catch-finally`.